

Отчёт по лабораторной работе №8

Реализация основных моделей в дискретно-событийном подходе

Сахно Никита Вячеславович

Abstract

В данном отчёте представлены результаты выполнения лабораторной работы по теме реализации стохастической SIR-модели в дискретно-событийном подходе на языке Julia.

Описаны использованные методы, структура программного комплекса, результаты базового моделирования и исследование влияния параметра вероятности заражения на динамику эпидемии.

Выполнено моделирование дискретно-событийной SIR-модели, сформированы временные ряды численности восприимчивых, инфицированных и переболевших, построены графики и выполнен запуск модели для набора значений параметра β .

Введение

Актуальность

Дискретно-событийный подход является одним из основных способов построения имитационных моделей, в которых состояние системы изменяется в моменты наступления отдельных событий. Такой подход позволяет описывать процессы, состоящие из большого количества индивидуальных взаимодействий, и естественным образом учитывать случайный характер этих взаимодействий.

В данной лабораторной работе дискретно-событийный подход применяется к классической SIR-модели распространения инфекции. В методических указаниях предлагается реализовать стохастическую модель в виде программного комплекса на языке Julia, выполнить моделирование, преобразовать код в литературный стиль и сформировать программные и документирующие представления результата.

Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR, реализовать стохастическую дискретно-событийную модель на языке Julia, выполнить её моделирование и исследовать влияние параметров на динамику системы.

Задачи

1. Создать рабочий каталог проекта и настроить окружение Julia.
2. Установить необходимые программные пакеты.
3. Реализовать ядро дискретно-событийной SIR-модели.
4. Реализовать скрипт запуска модели.
5. Выполнить базовый прогон модели с заданными начальными условиями.

6. Получить временные ряды численности групп S, I и R.
7. Построить и сохранить график динамики SIR-модели.
8. Преобразовать программу в литературный стиль с использованием Literate.jl.
9. Получить чистый Julia-код, Jupyter Notebook и документацию.
10. Выполнить исследование модели для набора значений параметра β .
11. Сравнить полученные результаты и сформулировать выводы.

Объект и предмет исследования

Объектом исследования является процесс распространения инфекции в конечной популяции.

Предметом исследования является динамика численности восприимчивых, инфицированных и переболевших индивидов в дискретно-событийной стохастической SIR-модели.

Методы исследования

В работе использованы методы имитационного моделирования, дискретно-событийного моделирования, генерации случайных величин, программной реализации на языке Julia, табличной обработки результатов и графической визуализации.

Для моделирования времени между контактами и времени до выздоровления используется экспоненциальное распределение. Управление виртуальным временем и очередь событий выполняется средствами ConcurrentSim.

Теоретическая часть

В работе используется классическая SIR-модель, в которой каждый индивид находится в одном из трёх состояний:

- S — восприимчивый к инфекции;
- I — инфицированный;
- R — переболевший и вышедший из процесса распространения инфекции.

В отличие от детерминированной модели, задаваемой системой обыкновенных дифференциальных уравнений, рассматриваемая модель является стохастической. Состояние системы изменяется при наступлении отдельных событий заражения и выздоровления.

Для восприимчивого индивида время до следующего социального контакта моделируется случайной величиной с экспоненциальным распределением:

$$T_c \sim \text{Exp}(c),$$

где c — средняя частота контактов.

При контакте с инфицированным индивидом заражение происходит с вероятностью β .

Для инфицированного индивида время до выздоровления моделируется экспоненциальным распределением с интенсивностью γ :

$$T_r \sim \text{Exp}(\gamma).$$

Таким образом, параметры модели имеют следующий смысл:

Параметр	Обозначение	Смысл
Вероятность передачи	β	Вероятность заражения восприимчивого при контакте с инфицированным
Частота контактов	c	Среднее число контактов одного индивида за единицу модельного времени
Интенсивность выздоровления	γ	Величина, обратная средней длительности болезни

Начальное состояние задаётся вектором

$$u_0 = [S_0, I_0, R_0].$$

Для базового эксперимента использованы значения

$$u_0 = [990, 10, 0],$$

то есть в популяции из 1000 человек первоначально 990 являются восприимчивыми, 10 — инфицированными и ни одного переболевшего нет.

В качестве параметров базового эксперимента использованы

$$p = [\beta, c, \gamma] = [0.05, 10.0, 0.25].$$

Дискретно-событийная реализация использует виртуальное время и очередь событий. Каждый индивид запускается как отдельный процесс, который может приостанавливаться до наступления следующего события. При заражении или выздоровлении соответствующие временные ряды обновляются.

Материалы и методы

В работе использовались:

- персональный компьютер;
- операционная система Windows;
- язык программирования Julia;
- пакетный менеджер Pkg;

- ConcurrentSim для дискретно-событийного моделирования;
- ResumableFunctions для возобновляемых процессов;
- Distributions для генерации случайных величин;
- DataFrames для формирования таблиц результатов;
- StatsPlots для построения графиков;
- DrWatson для организации проекта;
- Literate для преобразования литературного кода;
- Jupyter Notebook для выполнения сгенерированного ноутбука.

Рабочий каталог проекта:

```
C:\Users\nikit\Desktop\immod\lab08
```

Основные файлы проекта:

```
lab08/
├── src/
│   └── sir_model.jl
├── scripts/
│   └── sir_des.jl
├── literate/
│   ├── sir_des_literate.jl
│   └── sir_des_params_literate.jl
├── generated/
├── notebooks/
├── plots/
└── Project.toml
```

Базовый эксперимент выполнялся при

```
tmax = 40.0
u0 = [990, 10, 0]
p = [0.05, 10.0, 0.25]
```

Для исследования влияния параметра β использован набор

```
 $\beta$  = [0.03, 0.05, 0.07]
c = 10.0
 $\gamma$  = 0.25
```

Начальное состояние популяции для всех запусков оставалось неизменным.

Выполнение лабораторной работы

Создание и настройка проекта

Рабочий каталог лабораторной работы был создан по адресу:

```
C:\Users\nikit\Desktop\immod\lab08
```

В каталоге было создано Julia-окружение проекта. Для работы установлены пакеты BenchmarkTools, CSV, ConcurrentSim, DataFrames, Distributions, DrWatson, Literate, ResumableFunctions и StatsPlots.

Проверка окружения выполнялась командой:

```
using Pkg
Pkg.activate(".")
Pkg.status()
```

В результате было получено активное окружение проекта lab08 со всеми необходимыми зависимостями.

Реализация ядра SIR-модели

Основная логика модели размещена в файле:

```
src/sir_model.jl
```

Для хранения состояния отдельного индивида используется структура:

```
mutable struct SIRPerson
    id::Int64
    status::Symbol
end
```

Общее состояние модели хранится в структуре SIRModel:

```
mutable struct SIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end
```

Для обновления временных рядов при наступлении событий реализованы функции increment!, decrement! и carryover!.

При заражении выполняется функция:

```
function infection_update!(
    sim::ConcurrentSim.Simulation,
    m::SIRModel
)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ia)
    carryover!(m.Ra)
end
```

При выздоровлении используется:

```
function recovery_update!(  
    sim::ConcurrentSim.Simulation,  
    m::SIRModel  
)  
    push!(m.ta, ConcurrentSim.now(sim))  
    carryover!(m.Sa)  
    decrement!(m.Ia)  
    increment!(m.Ra)  
end
```

Жизненный цикл восприимчивого индивида реализован как дискретно-событийный процесс. Индивид ожидает случайное время до контакта, выбирает другого участника популяции и при контакте с инфицированным может перейти в состояние I.

Фрагмент реализации:

```
@resumable function live(  
    env::ConcurrentSim.Simulation,  
    individual::SIRPerson,  
    m::SIRModel  
)  
    while individual.status == :S  
        @yield timeout(env, rand(Exponential(1 / m.c)))  
  
        alter = individual  
  
        while alter == individual  
            N = length(m.allIndividuals)  
            index = rand(DiscreteUniform(1, N))  
            alter = m.allIndividuals[index]  
        end  
  
        if alter.status == :I  
            if rand(Uniform(0, 1)) < m.β  
                individual.status = :I  
                infection_update!(env, m)  
            end  
        end  
    end  
  
    if individual.status == :I  
        @yield timeout(env, rand(Exponential(1 / m.γ)))  
        individual.status = :R  
        recovery_update!(env, m)  
    end  
end
```

После определения модели были реализованы функции её создания, активации, запуска и получения результатов:

```

function MakeSIRModel(u0, p)
    (S, I, R) = u0
    N = S + I + R
    ( $\beta$ , c,  $\gamma$ ) = p

    sim = ConcurrentSim.Simulation()
    allIndividuals = SIRPerson[]

    for i = 1:S
        push!(allIndividuals, SIRPerson(i, :S))
    end

    for i = (S + 1):(S + I)
        push!(allIndividuals, SIRPerson(i, :I))
    end

    for i = (S + I + 1):N
        push!(allIndividuals, SIRPerson(i, :R))
    end

    ta = Float64[0.0]
    Sa = Int64[S]
    Ia = Int64[I]
    Ra = Int64[R]

    SIRModel(
        sim,  $\beta$ , c,  $\gamma$ ,
        ta, Sa, Ia, Ra,
        allIndividuals
    )
end

function activate(m::SIRModel)
    [
        @process live(m.sim, individual, m)
        for individual in m.allIndividuals
    ]
end

function sir_run(m::SIRModel, tf::Float64)
    ConcurrentSim.run(m.sim, tf)
end

function out(m::SIRModel)
    result = DataFrame()
    result[:, :t] = m.ta
    result[:, :S] = m.Sa
    result[:, :I] = m.Ia
    result[:, :R] = m.Ra
    return result
end

```

Базовый запуск модели

Для базового запуска использовался скрипт:

```
scripts/sir_des.jl
```

Параметры задавались следующим образом:

```
tmax = 40.0
u0 = [990, 10, 0]
p = [0.05, 10.0, 0.25]

Random.seed!(1234)

des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)
```

После завершения моделирования результат преобразовывался в DataFrame. Затем строился график временных рядов:

```
@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "Время",
    ylab = "Численность",
    title = "Дискретно-событийная SIR модель"
)

savefig(plotsdir("sir_des.png"))
```

График базового запуска сохранён в файл:

```
plots/sir_des.png
```

Преобразование программы в литературный стиль

Для выполнения требования методических указаний был создан литературный исходный файл:

```
literate/sir_des_literate.jl
```

В него были включены текстовые пояснения, формулы, программные фрагменты и выполняемые участки Julia-кода.

Для преобразования использовался пакет Literate:

```
using Literate

Literate.script(
    "literate/sir_des_literate.jl",
```

```

    "generated"
)

Literate.notebook(
    "literate/sir_des_literate.jl",
    "notebooks"
)

Literate.markdown(
    "literate/sir_des_literate.jl",
    "generated"
)

```

В результате были сформированы представления литературного кода в виде чистого Julia-файла, Jupyter Notebook и Markdown-документации.

Выполнение Jupyter Notebook

Сгенерированный файл:

```
notebooks/sir_des_literate.ipynb
```

был открыт в Jupyter Notebook и выполнен полностью.

В Notebook были выполнены ячейки и получены результаты базовой дискретно-событийной симуляции, а также построен график SIR-модели.

Исследование набора параметров

Для выполнения обязательной параметрической части был создан файл:

```
literate/sir_des_params_literate.jl
```

В нём исследуется влияние вероятности передачи инфекции β .

Использован набор:

```
betas = [0.03, 0.05, 0.07]
```

при фиксированных

```

c = 10.0
γ = 0.25

```

Для каждого значения β создавалась и запускалась отдельная модель:

```

for β in betas

    Random.seed!(1234)

    p = [β, c, γ]

    model = MakeSIRModel(u0, p)

```

```

    activate(model)

    sir_run(model, tmax)

    data = out(model)

    data[!, :β] .= β

    append!(results, data)

end

```

Для каждой серии экспериментов дополнительно определялась максимальная численность инфицированных и момент достижения максимума:

```

peak_results = DataFrame(
    β = Float64[],
    peak_I = Int64[],
    peak_time = Float64[]
)

for β in betas

    subset = filter(row -> row.β == β, results)

    index = argmax(subset.I)

    push!(
        peak_results,
        (
            β,
            subset.I[index],
            subset.t[index]
        )
    )

end

```

Для параметрической версии также был сформирован Jupyter Notebook:

`notebooks/sir_des_params_literate.ipynb`

Результаты

Результат базового моделирования

В результате базового запуска дискретно-событийной SIR-модели была получена таблица с колонками:

t
S
I
R

где t соответствует моменту наступления события, а S , I , R — текущей численности соответствующих групп.

Для базового эксперимента использовались:

```
tmax = 40.0  
u0 = [990, 10, 0]  
 $\beta = 0.05$   
 $c = 10.0$   
 $\gamma = 0.25$ 
```

На рис. 1 представлен полученный график временных рядов.

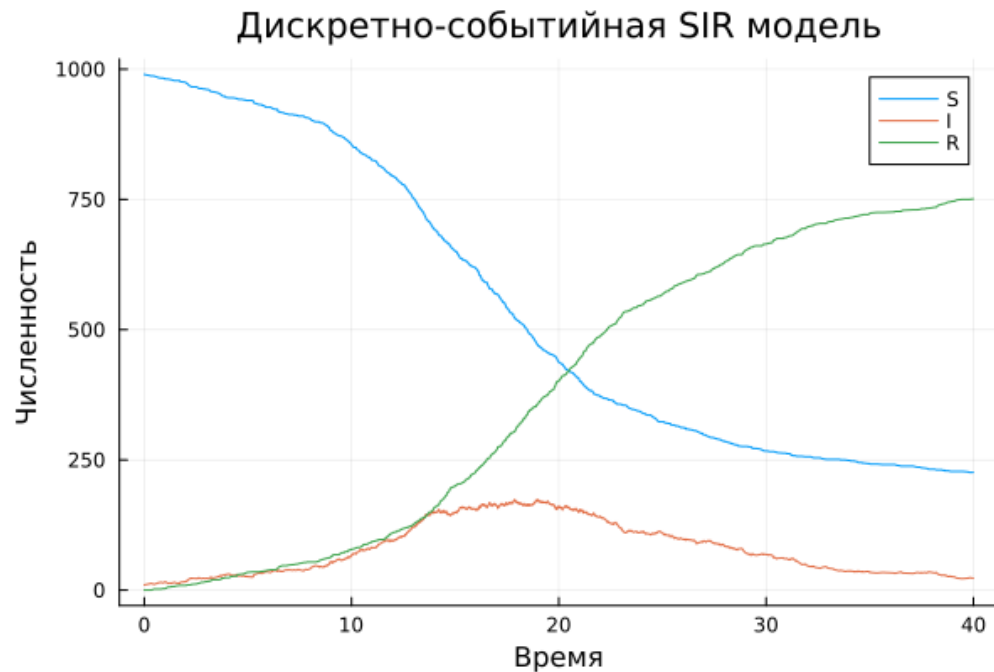


Figure 1: Результат базового запуска дискретно-событийной SIR-модели

Результат базового запуска дискретно-событийной SIR-модели.

Результат исследования набора параметров

Для параметрического эксперимента были рассмотрены три значения вероятности передачи инфекции:

```
 $\beta = 0.03$   
 $\beta = 0.05$   
 $\beta = 0.07$ 
```

При этом параметры ς и γ , а также начальное состояние популяции оставались неизменными.

На рис. 2 представлен сравнительный график для набора значений β .

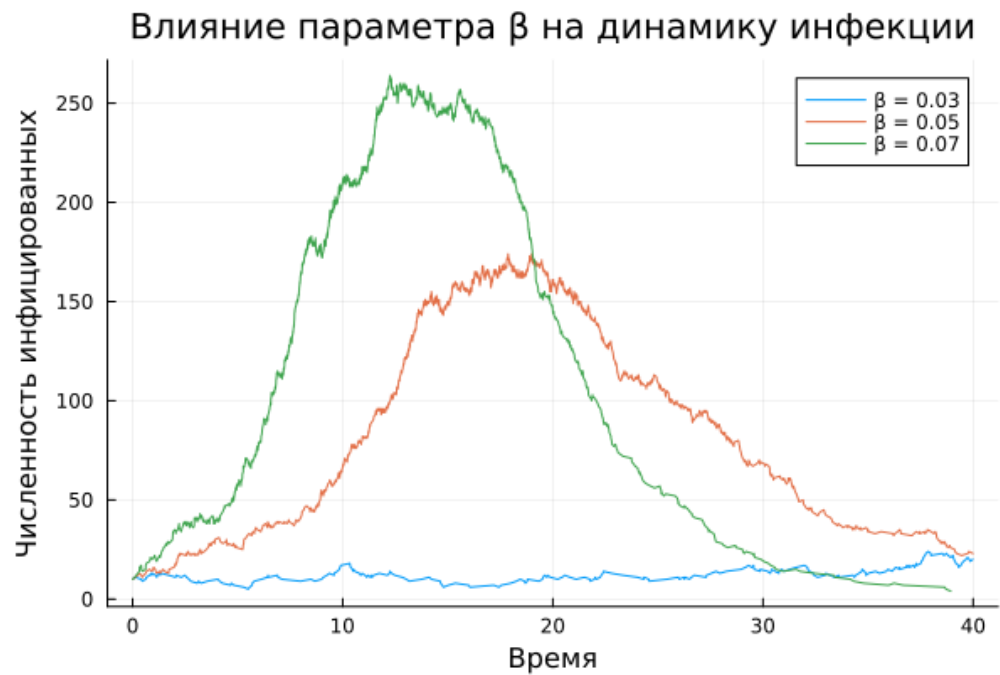


Figure 2: Сравнение динамики инфицированных при разных значениях β

Сравнение динамики численности инфицированных при различных значениях β .

В параметрическом эксперименте также формировалась таблица с показателями максимальной численности инфицированных и временем достижения максимума:

Параметр	Значения
β	0.03, 0.05, 0.07
ς	10.0
γ	0.25
Начальное состояние	[990,10,0]
Длительность моделирования	40.0

Обсуждение результатов

Полученная модель соответствует дискретно-событийному подходу: состояние каждого индивида изменяется не на каждом фиксированном шаге времени, а в момент наступления соответствующего случайного события.

При базовом запуске наблюдается изменение трёх групп популяции. Численность восприимчивых индивидов уменьшается по мере заражения, численность инфицированных изменяется вследствие одновременного появления новых случаев и

выздоровления, а численность переболевших увеличивается по мере выхода инфицированных из состояния болезни.

Важной особенностью модели являются случайные флуктуации. Они обусловлены случайным временем контактов, случайным выбором собеседника и вероятностным характером передачи инфекции. Поэтому траектория дискретно-событийной модели не является гладкой детерминированной кривой.

Изменение параметра β непосредственно влияет на вероятность заражения при контакте. При меньшем значении β распространение инфекции должно происходить медленнее, тогда как увеличение β повышает вероятность передачи инфекции и приводит к более интенсивной динамике распространения.

Параметрический запуск позволяет непосредственно сопоставить три варианта модели при одинаковых начальных условиях и неизменных значениях частоты контактов и интенсивности выздоровления. Полученный сравнительный график используется для визуальной оценки влияния β на число инфицированных.

Отдельно следует отметить воспроизводимость экспериментов. Перед запусками фиксировалось зерно генератора случайных чисел:

```
Random.seed!(1234)
```

Это позволяет повторять вычислительный эксперимент с одинаковой последовательностью псевдослучайных чисел при одинаковой структуре запуска.

С точки зрения программной реализации использование ConcurrentSim позволяет организовать виртуальное время и обработку событий, а ResumableFunctions — приостанавливать процессы индивидов до наступления следующего события.

Заключение

- Реализована стохастическая дискретно-событийная SIR-модель на языке Julia.
- Создано ядро модели с объектами SIRPerson и SIRModel.
- Реализованы события заражения и выздоровления и сбор временных рядов численности групп S, I и R.
- Выполнен базовый запуск модели при начальном состоянии [990,10,0] и параметрах $\beta = 0.05$, $c = 10.0$, $\gamma = 0.25$.
- Получен и сохранён график динамики дискретно-событийной SIR-модели.
- Литературный код преобразован с использованием Literate в чистый Julia-код, Jupyter Notebook и документацию.
- Выполнено исследование для набора значений $\beta = [0.03, 0.05, 0.07]$ при фиксированных $c = 10.0$ и $\gamma = 0.25$.
- Для параметрических прогонов рассчитаны максимальная численность инфицированных и момент достижения максимума.

- Поставленная цель лабораторной работы достигнута: освоена реализация дискретно-событийной SIR-модели и проведено исследование влияния параметра передачи инфекции.

Список литературы

Приложение А. Листинг ядра SIR-модели

Ниже приведён основной фрагмент программного кода, реализующий дискретно-событийную модель.

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using DataFrames
using Random

mutable struct SIRPerson
    id::Int64
    status::Symbol
end

mutable struct SIRModel
    sim::ConcurrentSim.Simulation
    β::Float64
    c::Float64
    γ::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

@resumable function live(
    env::ConcurrentSim.Simulation,
    individual::SIRPerson,
    m::SIRModel
)
    while individual.status == :S
        @yield timeout(env, rand(Exponential(1 / m.c)))

        alter = individual

        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
    end
end
```

```

        if alter.status == :I
            if rand(Uniform(0, 1)) < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end
    end
end

if individual.status == :I
    @yield timeout(env, rand(Exponential(1 / m.γ)))
    individual.status = :R
    recovery_update!(env, m)
end
end
end

```

Приложение Б. Параметрический эксперимент

Фрагмент кода, использованный для запуска серии экспериментов:

```

betas = [0.03, 0.05, 0.07]

results = DataFrame()

for β in betas

    Random.seed!(1234)

    p = [β, c, γ]

    model = MakeSIRModel(u0, p)

    activate(model)

    sir_run(model, tmax)

    data = out(model)

    data[:, :β] .= β

    append!(results, data)

end

```

Полученный сравнительный график сохраняется как:

plots/sir_beta_comparison.png

Приложение В. Формирование литературного кода

Для генерации программных и документирующих представлений использовался следующий код:

```
using Literate

Literate.script(
    "literate/sir_des_literate.jl",
    "generated"
)

Literate.notebook(
    "literate/sir_des_literate.jl",
    "notebooks"
)

Literate.markdown(
    "literate/sir_des_literate.jl",
    "generated"
)
```